Los Sabrossos

Contents

1 STRINGS

1.1 Aho corasick

```
//AhoCorasick para saber si un string s existe en un texto T
const int E = 26;
const int N = 1e6+10;
int cur;
int nodoTerminal[N];//se guarda el nodo terminal del string i
set<int>st;//se guarda que nodos son terminales que aparecen en el texto T

struct AC {
    int nodes;
    vector<int>suf, super;
    vector<array<int, E>>go;
    vector<bool>terminal;
    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
        super.emplace_back();
        suf.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
                else{
                    go[u][c] = u == 0 ? 0 :go[suf[u]][c];
                }
            }
        }
    }
};

```

```
void mark(int u, AC &ac){
    if(u == 0) return;
    mark(ac.super[u], ac);
    if(ac.terminal[u]){
        st.insert(u);
        ac.terminal[u] = false;
    }
    ac.super[u] = 0;
}

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t; //texto grande
    cin >> t;
    ac.build();
    int p = 0;
    for(auto x : t){
        int c = x - 'a';
        p = ac.go[p][c];
        mark(p, ac);
    }
    repl(i,0,n){
        if(st.count(nodoTerminal[i])){
            cout << "YES\n";
        }
        else{
            cout << "NO\n";
        }
    }
}

//------------------------------------------------------------------------

//AhoCorasick para Frecuencias
const int E = 26;
const int N = 1e6+10;
int nodoTerminal[N];
int cur;
struct AC {
    int nodes;
    vector<int>suf, freq, by_level;
    vector<bool>terminal;
    vector<array<int,E>>go;

    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        suf.emplace_back();
        go.emplace_back();
        freq.emplace_back();
    }

```

```
    void insert(string &s){
        int pos = 0;
        for(char ch:s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] = true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            by_level.emplace_back(u);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
                else{
                    go[u][c] = u == 0 ? 0 : go[suf[u]][c];
                }
            }
        }
    }
};

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t;
    cin >> t;
    ac.build();
    int p = 0;
    for(char ch:t){
        int c = ch-'a';
        p = ac.go[p][c];
        ac.freq[p]++;
    }
    for(int i = (int)ac.by_level.size()-1; i > 0; i--){
        int x = ac.by_level[i];
        ac.freq[ac.suf[x]] += ac.freq[x];
    }
    repl(i,0,n){
        int termi = nodoTerminal[i];
        cout << ac.freq[termi] << '\n';
    }
}
```

1.2 Hashing

```
struct Hash {
  long long P=1777771,MOD[2],PI[2];
  vector<long long> h[2],pi[2];
  Hash(string& s){
    MOD[0]=999727999;MOD[1]=1070777777;
    PI[0]=325255434;PI[1]=10018302;
    for(int k = 0; k < 2; k++) h[k].resize(s.size()+1),pi[k].resize(s.size()+1);
    for(int k = 0; k < 2; k++){
      h[k][0]=0;pi[k][0]=1;
      long long p=1;
            for(int i = 1; i < s.size()+1; i++){
        h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
        pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
        p=(p*P)%MOD[k];
      }
    }
  }
  long long get(int s, int e){
    long long h0=(h[0][e]-h[0][s]+MOD[0])%MOD[0];
    h0=(1LL*h0*pi[0][s])%MOD[0];
    long long h1=(h[1][e]-h[1][s]+MOD[1])%MOD[1];
    h1=(1LL*h1*pi[1][s])%MOD[1];
    return (h0<<32)|h1;
  }
};
```

1.3 Kmp

```
vector<int> kmp(string s){
    int n = s.size();
    vector<int> pi(n);
    for(int i = 1; i < n; i++){
        int j = pi[i-1];
        while(j > 0 and s[i] != s[j]){
            j = pi[j-1];
        }
        if(s[i] == s[j]){
           j++;
        }
        pi[i] = j;
    }
    return pi;
}
//Pattern Matching
vector<int> ocurrencia(string texto, string patron) {
    vector<int> P = kmp(patron);
    int k = 0;
    vector<int> M;
    repl(i,0,texto.size()) {
        while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
        if (patron[k] == texto[i]) k++;
        if (k == patron.size()) {
            M.pb(i - k + 1);
            k = P[k - 1];
        }
    }
    return M;
}
```

```cpp
//Automata
void genAut(string &s) {
    int n = s.size();

    vector<vector<int>> aut(n+1, vector<int>(26));
    vector<int> pi = kmp(s);

    for(int i = 0 ; i < n; i++){
        aut[i][s[i]-'a'] = i+1;
    }

    for(int i = 0; i < n+1; i++){
        for(int c = 0; c < 26; c++){
            if (aut[i][c]) continue;
            if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
            else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
            else aut[i][c] = 0;
        }
    }
}
```

1.4 Suffix array

```cpp
vector<int> suffix_array(string s) {
    s += char(0);
    const int n = s.size();
    vector<int>a(n);
    iota(a.begin(), a.end(), 0);
    sort(a.begin(), a.end(), [&] (int i, int j){
        return s[i] < s[j];
    });
    vector<int> mapping(n);
    mapping[a[0]] = 0;
    for(int i = 1; i < n; i++){
        mapping[a[i]] = mapping[a[i-1]]+(s[a[i-1]] != s[a[i]]);
    }
    int len = 1;
    vector<int>sbs(n);
    vector<int>head(n);
    vector<int>new_mapping(n);
    while(len < n){
        for(int i = 0; i<n; i++) sbs[i] = (a[i] + n - len) % n;
        for(int i = n-1; i>=0; i--) head[mapping[a[i]]] = i;
        for(int i = 0; i < n; i++){
            int x = sbs[i];
            a[head[mapping[x]]++] = x;
        }
        new_mapping[a[0]] = 0;
        for(int i = 1; i < n; i++){
            if(mapping[a[i-1]] != mapping[a[i]]){
                new_mapping[a[i]] = new_mapping[a[i-1]]+1;
            }
            else{
                int pre = mapping[(a[i-1] + len)%n];
                int cur = mapping[(a[i]+len) % n];
                new_mapping[a[i]] = new_mapping[a[i-1]] + (pre!=cur);
            }
        }
        len <<= 1;
        swap(mapping, new_mapping);
    }
    return vector<int>(a.begin()+1, a.end());//ignorar a[0] que es el centinela
```

```
}

vector<int> lcp_array(vector<int>&a, string s){
    const int n = s.size();
    vector<int> rank(n);
    for(int i = 0; i < n; i++) rank[a[i]] = i;
    int k = 0;
    vector<int>lcp(n);
    for(int i = 0; i < n; i++){
        if(rank[i] == n-1){
            k = 0;
            continue;
        }
        int j = a[rank[i]+1];
        while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[rank[i]] = k;
        if(k) k--;
    }
    lcp.pop_back();
    return lcp;
}

void solve(){
    //Numero de distintos substr en un string n*(n+1)/2 - sumatoria del LCP
    //verificar si un string es substring de otro con BS en SA
    //encontrar el longest common substring en 2 strings(ESTA IMPLEMENTACION)
    string s, t;
    cin >> s >> t;
    if(s.size() > t.size()){
        swap(s, t);
    }
    int n = s.size();
    int m = t.size();
    string st = s+'#'+t;
    vector<int>sa = suffix_array(st);
    vector<int>lcp =lcp_array(sa, st);
    // print(sa);
    // print(lcp);
    vector<int>tr(n+m);
    repl(i,0,n+m){
        if(sa[i] >=n and sa[i+1]>=n){
            tr[i] = 0;
            continue;
        }
        if(sa[i] < n and sa[i+1] < n){
            tr[i] = 0;
            continue;
        }
        int mn = min(sa[i], sa[i+1]);
        int tam1 = mn + lcp[i];
        // dbg(tam1);
        tr[i] = lcp[i];
        if(tam1 > n){
            dbg(tam1);
            tr[i] = lcp[i] - (tam1-n);
        }
    }
    // print(tr);
    int mx = *max_element(tr.begin(), tr.end());
    repl(i,0,n+m){
        // dbg(tr[i]);
        if(tr[i] == mx){
            string ans = st.substr(sa[i], mx);
```

```
            cout << ans << '\n';
            return;
        }
    }

}
//Sabrossus
```

1.5 Trie

```
const int E = 26;

struct AC {
    int nodes;
    vector<array<int, E>>go;
    vector<bool>terminal;

    AC(){
        add_node();
        nodes = 1;
    }

    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - '0';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
    }

};
```

2 GRAPH ALGORITHMS

2.1 Dijkstra

```
const int N = 1e5;

int vis[N];
vector<pair<int,int>> G[N];

void dijkstra(int x){
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0,x});
    vis[x] = 0;
    while(!pq.empty()){
        auto y = pq.top();
        pq.pop();
        int u = y.second;
```

```
        int cost = y.first;
        for(auto v : G[u]){
            int cur = cost + v.second;
            if(cur < vis[v.first]){
                vis[v.first] = cur;
                pq.push({cur,v.first});
            }
        }
    }
}
```

2.2 Kruskal mst

```
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x]+=sizes[y];
        }
    }
};
int n;
vector<pair<int,pair<int,int>>> List;
int Kruskal_MST(){
    sort(List.begin(),List.end());
    DSU dsu(n);
    int mst = 0, t = 1;
    for(int i = 0; i < List.size(); i++){
        int w = List[i].first; //weigth
        int a = List[i].second.first; //at
        int b = List[i].second.second; //to
        if(dsu.find(a) != dsu.find(b)){
            dsu.join(a,b);
```

```
            mst+=w;
            t++;
        }
        if(t == n) break;
    }
    return mst;
}
```

2.3 Topological sort kahn algorithm

```
const int N = 1e5;

int indegree[N];
vector<int> G[N];
int n;
vector<int> Kahn(){
    queue<int> q;
    vector<int> ans;
    for(int i = 0; i < n; i++){
        if(indegree[i] == 0) q.push(i);
    }
    while (!q.empty()){
        int v = q.front();
        q.pop();
        ans.push_back(v);
        for(int u : G[v]){
            indegree[u]--;
            if(indegree[u] == 0){
                q.push(u);
            }
        }
    }
    //if(ans.size() != n) grafo no DAG
    return ans;
}
```

3 MATH

3.1 Binary exponentiation

```
long long pote(long long a, long long b, long long mod){
    long long ans = 1ll;
    while(b){
        if(b&1) ans = (ans * a)%mod;
        a = (a * a)%mod;
        b>>=1;
    }
    return ans;
}
// a^-1 = a^mod-2 = pote(a,mod-2);
```

3.2 Modular inverse

```
const long long N = 1e6 + 10;
const int MOD = 1e9+7;
long long fact[N];
long long invfact[N];
```

```
void init(){
    fact[0] = 1;
    for(long long i = 1; i < N; i++)
        fact[i] = fact[i-1] * i % MOD;

    invfact[N-1] = pote(fact[N-1], MOD-2);

  for(long long i = N-2; i >= 0; i--)
        invfact[i] = invfact[i+1] * (i+1) % MOD;
}

long long nCk(long long n, long long k){
    if(k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
}
```

3.3 Pollard rho rabin miller

```
map<long long, long long> fact;

long long mcd(long long a, long long b) {
    a = abs(a);
    b = abs(b);
    while (a != 0) {
        long long r = b % a;
        b = a;
        a = r;
    }
    return b;
}
//Rabin Miller
bool primo(long long n) {
    if (n < 2) return false;
    if (n == 2) return true;
    long long D = n - 1, S = 0;
    while (D % 2 == 0) {
        D /= 2;
        S++;
    }
    static const long long STEPS = 16ll;
    for(long long pasos = 0; pasos < STEPS; pasos++){
        const long long A = 1 + rand() % (n - 1);
        long long M = pote(A, D, n);
        if (M == 1 || M == (n - 1)) goto next;
        for(long long k = 0; k < S-1; k++){
            M = ((M) * M) % n;
            if (M == (n - 1)) goto next;
        }
        return false;
    next:;
    }
    return true;
}

//Rho de pollard
long long factor(long long n) {
    static long long A, B;
    A = 1 + rand() % (n - 1);
    B = 1 + rand() % (n - 1);
    #define fun(x) (((x) * (x + B)) % n + A)

    long long x = 2, y = 2, d = 1;
```

```cpp
    while (d == 1 || d == -1) {
        x = fun(x);
        y = fun(fun(y));
        d = mcd(x - y, n);
    }
    return abs(d);
}

void factorize(long long n){
    assert(n > 0);
    while (n > 1 && !primo(n)) {
        long long fx;
        do {
            fx = factor(n);
        } while (fx == n);

        n /= fx;
        factorize(fx);

        for (auto &it : fact) {
            while (n % it.first == 0) {
                n /= it.first;
                it.second++;
            }
        }
    }
    if (n > 1) fact[n]++;
}
```

3.4 Sieve of eratosthenes

```cpp
const int N = 1e7;
int sieve[N];
void make(){
    for(int i = 0; i < N; i++) sieve[i] = 1;
    sieve[0] = sieve[1] = 0;
    for (int i = 2; i < N; i++)
    {
        if(sieve[i]){
            for (int j = i*i; j < N; j+=i)
            {
                sieve[j] = 0;
            }
        }
    }
}
```

4 DATA STRUCTURES

4.1 Disjoint set union

```cpp
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
```

```
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x]+=sizes[y];
        }

    }
};
```

4.2 Ordered set

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
    ordered_set;

void solve() {
    ordered_set st;

    st.insert(10);
    st.insert(30);
    st.insert(20);

    // 1. Elemento en la posicion K (0-indexed, de menor a mayor)
    // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
    auto it = st.find_by_order(1);
    if (it != st.end()) {
        int elemento = *it; // Devuelve 20 de forma segura
    }

    // 2. Cuantos elementos son estrictamente menores que X
    int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
}
// Sabrossus
```

4.3 Segment tree

```
const int N = 1e5;
struct info{
    int num;
};
info ST[4*N];
info ope(info u, info v){
    return {u.num+v.num};
}
void build(int in, int L, int R){
    if(L == R){
        cin >> ST[in].num;
    }
    else{
        int mid = (L+R)>>1;
        build((2*in),L,mid);
        build((2*in)+1,mid+1,R);
        ST[in] = ope(ST[(2*in)], ST[(2*in)+1]);
    }
}
void update(int in, int L, int R, int pos, int val){
    if(L == R){
        ST[in].num = val;
    }
    else{
        int mid = (L+R)>>1;
        if(pos <= mid){
            update(2*in,L,mid,pos,val);
        }
        else{
            update((2*in)+1,mid+1,R,pos,val);
        }
        ST[in] = ope(ST[2*in], ST[(2*in)+1]);
    }
}
info get(int in, int L, int R, int l, int r){
    if(r < L or R < l) return {0};
    if(l <= L and R <= r) return ST[in];
    int mid = (L+R)>>1;
    info left = get(2*in,L,mid,l,r);
    info right = get((2*in)+1,mid+1,R,l,r);
    return ope(left,right);
}
// v = {1 2 3 4 5}
// get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
```

4.4 Sparse table

```
const int N = 1e5;
const int LOG = 20;
int v[N]; //values
int st[N][LOG]; //Sparse Table
int lg[N]; // log values
int n;
void build(){
    for(int i = 0; i < n; i++) st[i][0] = v[i];

    for(int i = 1; i < LOG; i++){
        for(int j = 0; j < n - (1<<i) + 1; j++){
            st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
```

```
        }
    }
    lg[1] = 0;
    for(int i = 2; i <= n;i++) lg[i] = lg[i/2] + 1;
}
int query(int L, int R){
    int k = lg[R-L+1];
    return min(st[L][k],st[R-(1<<k)+1][k]);
}
// v = [5, 2, 4, 7, 1, 3]
// query(1, 4) -> [2, 4, 7, 1]
```

5 FLOW

5.1 Dinic

```
struct Dinic
{
    const int INF = 1e9;

    struct flowEdge
    {
        int to, rev, f, cap;
    };

    vector<vector<flowEdge>> G;
    vector<int> work, lvl, Q;

    int S, T, nodes;

    Dinic(){}
    Dinic(int n, int s, int t){
        S = s;
        T = t;
        nodes = n;
        G.clear();
        G.resize(n);
        Q.resize(n);
    }
    void addEdge(int st, int en , int cap){
        flowEdge A = {en, (int)G[en].size(), 0, cap};
        flowEdge B = {st, (int)G[st].size(), 0, 0};
        G[st].pb(A);
        G[en].pb(B);
    }
    int dfs(int v, int f){
        if(v == T or f == 0) return f;
        for(int &i = work[v]; i < G[v].size(); i++){
            flowEdge &e = G[v][i];
            int u = e.to;
            if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
            int df = dfs(u, min(f, e.cap - e.f));
            if(df){
                e.f += df;
                G[u][e.rev].f -= df;
                return df;
            }
        }
        return 0;
    }
    bool bfs(){
```

```
        int qt = 0;
        Q[qt++] = S;
        lvl.assign(nodes,-1);
        lvl[S] = 0;
        for(int i = 0; i < qt; i++){
            int u = Q[i];
            for(flowEdge &e : G[u]){
                int v = e.to;
                if(e.cap <= e.f or lvl[v] != -1) continue;
                lvl[v] = lvl[u] + 1;
                Q[qt++] = v;
            }
        }
        return lvl[T] != -1;
    }
    int maxFlow(){
        int flow = 0;
        while(bfs()){
            work.assign(nodes,0);
            while(true){
                int df = dfs(S,INF);
                if(df == 0) break;
                flow += df;
            }
        }
        return flow;
    }

};
```

6 UTILITIES

6.1 Merge sort

```
// Rango de uso: 0-indexed.
// Llamar en el solve como: mergeSort(v, 0, n - 1);
ll ans = 0;

void mergeSort(vector<ll> &v, int low, int high) {
    if (low == high) return;

    int mid = (low + high) >> 1;
    mergeSort(v, low, mid);
    mergeSort(v, mid + 1, high);

    queue<ll> L, H;
    for (int i = low; i < mid + 1; i++) {
        L.push(v[i]);
    }
    for (int i = mid + 1; i < high + 1; i++) {
        H.push(v[i]);
    }

    for (int i = low; i < high + 1; i++) {
        if (L.size() == 0) {
            v[i] = H.front();
            H.pop();
        }
        else if (H.size() == 0) {
            v[i] = L.front();
            L.pop();
```

```
        }
        else {
            if (L.front() <= H.front()) {
                v[i] = L.front();
                L.pop();
            }
            else {
                v[i] = H.front();
                H.pop();
                ans += L.size(); // Inversion detectada
            }
        }
    }
}
```

7 GEOMETRY

7.1 Convex hull

```
// Devuelve sentido horario
vector<point> hull(vector<point> p)
{
    int n = p.size();
  vector<point> h;
  sort(p.begin(),p.end());
  for(int i = 0; i < n; i++){
    while(h.size() >= 2 and p[i].left(h[h.size () - 2], h.back()) < 0){
      h.pop_back();
    }
    h.push_back(p[i]);
  }
  h.pop_back();
  int k = h.size();
  for(int i = n-1; i >= 0; i--)
  {
    while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
      h.pop_back();
    }
    h.pb(p[i]);
  }
  h.pop_back();
  return h;
}
```

7.2 Point

```
struct point
{
  int x, y;
  point() {}
  point(int x, int y): x(x), y(y) {}
  point operator -(point p) {return point(x - p.x, y - p.y);}
  point operator +(point p) {return point(x + p.x, y + p.y);}
  int sq() const{return x * x + y * y;}
  double abs() const {return sqrt(sq());}
  int operator ^(point p) {return x * p.y - y * p.x;}
    int operator *(point p) {return x * p.x + y * p.y;}
    point operator *(int a) {return point(x * a, y * a);}
  bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
```

```
  int left(point a, point b) {
        return ((b - a) ^ (*this - a)); // 0, -, +
    }
};
ostream& operator<<(ostream& os, const point& p) {
    return os << "(" << p.x << "," << p.y << ")\n";
}


void polarSort(vector<point>& v) {
  sort(v.begin(), v.end(), [] (point a, point b) {
    const point origin{0, 0};
    bool ba = a < origin, bb = b < origin;
    if (ba != bb) { return ba < bb; }
    return (a^b) > 0;
  });
}
```

8 TREE ALGORITHMS

8.1 Binary lifting

```
const int N = 1e5;
const int LOG = 20;
int up[N][LOG];
vector<int> G[N];
//Find parents
void dfs(int u){
    for(int v : G[u]){
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_K_parent(int a, int k){
    for(int j = LOG-1; j >= 0; j++){
        if(k&(1<<j)){
            a = up[a][j];
        }
    }
    return a;
}
```

8.2 Euler tour

```
const int N = 1e5 + 10;
vector<int> path;
vector<int> G[N];
void EulerTour(int node, int parent){
    path.pb(node);
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u,node);
        }
    }
    path.pb(node);
}
```

```cpp

const int N = 1e5 + 10;
int tin[N];
int tout[N];
int flat[N];
int timer = 1;
void EulerTour(int node, int parent){
    tin[node] = timer;
    flat[timer] = node;
    timer++;
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u, node);
        }
    }
    tout[node] = timer;
}
```

8.3 Lowest common ancestor

```cpp
const int N = 1e5;
const int LOG = 20;
vector<int> G[N];
int depth[N];
int up[N][LOG];
// Assign levels and get parents
void dfs(int u){
    for(int v : G[u]){
        depth[v] = depth[u] + 1;
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_lca(int a, int b){
    if(depth[a] < depth[b]) swap(a,b);
    int k = depth[a] - depth[b];
    for(int j = LOG-1; j >= 0; j--){
        if(k & (1<<j)){
            a = up[a][j];
        }
    }
    if(a == b) return a;
    for(int j = LOG-1; j >= 0; j--){
        if(up[a][j] != up[b][j]){
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
```

9 BIT MANIPULATION

9.1 Binary representation

```
string tobit(long long x) {
    string a(64, '0');
    for (int i = 63; i >= 0; --i) {
        if (x & 1) a[i] = '1';
        x >>= 1;
    }
    return a;
}
```